

CreditSetu: A Dual-Path Credit Scoring & Lending Marketplace for Microfinance

Prototype Stage Report for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Maanya Garg, CSED* Kashvi Bansal, CSED[†] Shreesh Gupta, CSED[‡]
Shivam Raj, CSED[§]

September 17, 2026

Abstract

CreditSetu is a dual-path credit-scoring and lending-marketplace platform for microfinance in India. Its core proposition is that every borrower builds their *own* credit score from their own repayment history, while a borrower who belongs to a Self-Help Group (SHG) gets a trust-based bootstrap head start from their group’s track record and an independent borrower starts from a flat base and builds purely from scratch. Scores are produced by an XGBoost classifier and explained feature-by-feature with SHAP, so a borrower or lender sees not just a number but why it is what it is. Lenders formally partner with specific SHGs through an approve/reject linkage rather than open lending to anyone, and a loan marketplace lets lenders make offers and borrowers request loans directly. At the prototype stage, the system has been rebuilt around four separate, role-gated logins (Borrower, Lender, SHG, Admin), each authorized server-side and scoped strictly to that role’s own data, and extended with a live borrower repayment flow (a wallet balance and a dummy payment gateway) so a borrower’s actions – not just offline synthetic history – move their own score. This report covers the system’s architecture, data model, scoring methodology, feasibility, what has actually been built and verified working, and the accompanying UML design diagrams.

Keywords: Microfinance, Credit Scoring, XGBoost, SHAP, Explainable AI, Self-Help Groups, Role-Based Access Control, FastAPI, React.

1 Introduction

... *background and motivation*

Microfinance lending in India serves two kinds of borrowers very unevenly. A member of a Self-Help Group (SHG) benefits from the group’s collective track record and social accountability, while an independent borrower with no group affiliation and no prior loan has nothing a lender can score against – the classic cold-start problem. Where informal scoring does exist, it is rarely explainable: a borrower or lender sees a number with no account of *why* it is what it is, which undermines both borrower trust and lender diligence. Structured, auditable relationships

*Roll No: 1024030564

[†]Roll No: 1024030563

[‡]Roll No: 1024030565

[§]Roll No: 1024031140

between a specific lender and a specific SHG are also usually absent – a lender either serves an entire group’s members or none of them, with no formal, revocable agreement in between.

An earlier version of this project (**Sahara MFI**, a Node/Express + MySQL application) addressed SHG-only scoring through a hand-written SQL formula, with no independent-borrower path, no machine-learning component, and no lender marketplace. CreditSetu is a from-scratch rebuild: the general domain (SHGs, loans, repayments, credit scores) carries over, but the schema, scoring approach, and system design are new, driven by the dual-path requirement that did not fit the old single-path schema.

1.1 Problem Statement

- **Cold-start scoring:** an independent borrower with no formal credit history cannot be scored by a model trained only on borrowers who already have repayment history.
- **Opaque scoring:** even where a score exists, it is rarely explained in a way a borrower or lender can act on.
- **No formal SHG-lender relationship:** lending decisions at group level are informal and unauditable.
- **Undifferentiated access:** in an open system, a borrower, a lender, and an SHG all see the same data, with no role-appropriate, access-controlled view for any of them.

1.2 Objectives

- Build a dual-path scoring engine that gives SHG members a fair, group-earned head start while letting independent borrowers build a score purely from their own behaviour.
- Make every score explainable, feature by feature, using SHAP.
- Formalize the SHG-lender relationship as an explicit approve/reject linkage.
- Separate the system into four role-gated views (Borrower, Lender, SHG, Admin), each authorized server-side, so no role can see another’s data.
- Let a borrower’s own actions – not just offline synthetic history – move their own score, via a live repayment flow.

2 Related Work

... existing approaches

Traditional microfinance credit assessment relies heavily on group liability and manual field-agent judgement, with limited use of statistical or machine-learning scoring. Where formal credit bureaus exist (e.g. CIBIL in India), they score individuals with an existing formal credit history and have no mechanism for a first-time, group-affiliated borrower to inherit a trust signal from their group. Commercial microfinance information systems (including the team’s own earlier **Sahara MFI** build) typically implement a single, SHG-only scoring formula with no cold-start path for independent borrowers and no per-feature explanation of the resulting score. Explainable-AI techniques such as SHAP [1] have seen adoption in consumer credit scoring in developed markets, but are less common in microfinance-specific tooling, where model opacity and the cold-start problem are usually treated as separate, unrelated concerns rather than solved together through an explicit bootstrap-to- model blend, as CreditSetu does.

3 Proposed Methodology

... *system design*

3.1 System Architecture

CreditSetu is a 3-tier web application:

- **Frontend:** a React (Vite) single-page application with four role-gated dashboards (Borrower, Lender, SHG, Admin) and a public landing page, styled as a government-portal-style demonstration platform (clearly labelled as such, not impersonating any real government property).
- **Backend API:** FastAPI, exposing REST endpoints under `/api`, with bearer-token session authentication per role and server-side authorization scoping every query to the authenticated caller's own identity.
- **Database:** SQLAlchemy over SQLite for the demonstration deployment (a one-line `DATABASE_URL` environment-variable change moves this to Postgres for concurrent production use, with no code changes).
- **ML pipeline:** offline scripts (not imported by the API at request time) for synthetic data generation, model training, and batch scoring, producing artifacts the API serves.

3.2 Data Model

The schema has thirteen core tables plus the role-based-access and payments additions made during this iteration: `districts`, `shgs`, `individuals` (the dual-path pivot – `shg_id` is nullable, and NULL means independent), plus per-role `...sessions` tables (individual, lender, shg, admin), `shg_attendance_records`, `savings_records`, `lenders`, `shg_lender_links` (the approve/reject table, extended with an `initiated_by_role` column so the UI can correctly determine which side may approve/reject a given link), `loans` (extended with a borrower-chosen `target_lender_id` on the request side), `repayment_events`, `credit_scores` (append-only score history, not a single mutable column), `score_explanations` (per-feature SHAP contribution, in score points, per score), `loan_offers` and `loan_requests` (the two-directional marketplace flow), and `anomaly_flags`. The full class-level detail of every table, field, and relationship is given in the Class Diagram in Section B.

3.3 Scoring Algorithm

Input:

- An individual's loans, repayment events, savings records, and (if SHG-linked) attendance records.
- Trained XGBoost classifier and its SHAP `TreeExplainer`.
- Configured bootstrap parameters: `SCORE_MIN`, `SCORE_MAX`, `INDEPENDENT_BASE_SCORE`, the SHG peer-trust bootstrap formula, and `HISTORY_MATURITY_EVENTS`.

Output: a score in `[SCORE_MIN, SCORE_MAX]`, a risk category, the scoring basis (`shg_bootstrap / independent_bootstrap / model`), and a per-feature SHAP explanation summing to `score - SCORE_MIN`.

1. Turn the individual's raw history into ~ 16 engineered features (repayment rate, savings regularity, SHG tenure, peer repayment rate, etc.); a missing input is represented as NaN, never silently zeroed.
2. Count `n_repayment_events` for this individual.

3. If zero: return the pure prior – the SHG bootstrap (derived from the group’s own aggregate repayment/attendance rate) for an SHG-linked individual, or the flat `INDEPENDENT_BASE_SCORE` for an independent one. No model call.
4. If between 1 and `HISTORY_MATURITY_EVENTS`: linearly blend the prior and the trained model’s prediction, weighted by how much real history has accumulated.
5. If at or past `HISTORY_MATURITY_EVENTS`: use the model’s prediction alone.
6. Run the SHAP `TreeExplainer` on the model’s output (or, for a pure-prior score, synthesize an explanation anchored on the model’s own baseline prediction rather than the floor, so a high-risk borrower’s explanation does not collapse to near-empty – see Section 5.4).
7. Persist the score as a new, append-only `CreditScore` row and its `ScoreExplanation` rows.

3.4 Role-Based Access Control

Four roles – Borrower, Lender, SHG, Admin – each log in separately (phone+password for borrowers, username+password for the other three, all demo-seeded with the same hashing scheme, no JWT). Every endpoint in the API requires the correct role’s bearer token and derives ”my own data” from the session, never from a client-supplied id; an out-of-scope request is rejected with 401 (no/bad token) or 403 (wrong role or wrong identity), enforced server-side rather than only hidden in the UI. Table 1 summarises what each role may see.

Role	Sees
Borrower	Own score/history/SHAP explanation/improvement tips, own SHG’s public stats, own loans, own requests/offers, the full lender directory (read-only).
Lender	SHGs browsable by district, member lists for SHGs they are linked to, incoming loan requests, own reputation score.
SHG	Own members and stats, own linked/pending lender links.
Admin	Read-only, platform-wide: all borrowers/SHGs/lenders, district stats, anomaly flags, model health.

Table 1: Role → data-access matrix

3.5 Repayment and Payment Gateway

A borrower can direct a loan request at a specific eligible lender (in addition to the existing broadcast-to-all-eligible-lenders behaviour), with a retarget option if a targeted request is declined. Once a loan is active, ”amount due this cycle” is derived from the count of existing repayment events on that loan (not from elapsed calendar time), so a loan is payable the instant it is disbursed rather than only after a real due date has passed – necessary for the feature to be usable by a live user rather than only replayable against backdated data. A dummy payment gateway (clearly labelled as non-real) shows the borrower’s wallet balance and the amount due per active loan; if the wallet cannot cover the borrower’s *total* dues across all active loans this cycle, payment of every loan is blocked, not just the one clicked. A successful payment creates a real `RepaymentEvent`, updates the loan’s outstanding balance and status, and recomputes the credit score in the same transaction.

4 Feasibility Report

... can this actually be built and run

4.1 Technical Feasibility

Every component uses mature, well-documented open-source tooling already proven at this scale: FastAPI and SQLAlchemy for the API and ORM, XGBoost and SHAP for scoring and explanation, scikit-learn for k-means district clustering and isolation-forest anomaly detection, and React/Vite for the frontend. No component requires novel research or unavailable infrastructure. The team has already built, run, and fixed real defects in this stack (see Section 5.4), demonstrating the architecture is technically sound in practice, not just on paper.

4.2 Operational Feasibility

The system runs on a single machine for demonstration (SQLite, `uvicorn --reload`, `npm run dev`) with no external paid service dependency. A production deployment would swap SQLite for a managed Postgres instance via a single environment variable and add a proper secrets/password-hashing setup in place of the deliberately minimal demo scheme – both are additive changes, not architectural rewrites.

4.3 Economic Feasibility

All tooling used is free and open-source; there is no licensing cost. The only cost at production scale would be standard cloud hosting (a database instance and a small API/web server), which is proportionate to any comparable small-to-medium SaaS deployment and not a blocker for a course project or an early pilot.

4.4 Schedule Feasibility

The 8-stage plan (synthetic data generation, scoring engine, cold-start bootstrap, SHG-lender linkage, lender matching/offers, frontend, geo-clustering, anomaly detection) was completed and verified end-to-end before this iteration began. The current iteration – role separation, server-side authorization, targeted requests, and the live repayment flow – was completed, tested against the live API and browser, and is documented with real before/after verification (Section 5), indicating the remaining course milestones (further hardening, the Final Report) are reachable within the semester timeline.

5 Implementation and Results

... what actually works, verified

All figures in this section are measured against the live system, not estimated.

5.1 Current Scale

The seeded dataset spans 1,079 individuals (659 SHG-linked, 420 independent), 51 SHGs, 6 lenders, and 10 districts, with over 1,600 loans and their associated repayment events.

5.2 Verified End-to-End: Loan Lifecycle and Score Impact

A real borrower account was taken through the full live flow with no manual database edits: request a loan → have it approved by a real lender login → repay it through the payment gateway. The credit score moved from 596 (Medium risk) to 886 (Very Low risk) as real on-time repayment history crossed `HISTORY_MATURITY_EVENTS`, with the scoring basis switching from `independent_bootstrap` to `model` – exactly the behaviour the methodology in Section 3.3 specifies.

5.3 Verified: Access Control

Cross-role and cross-identity requests were tested directly against the live API (e.g. a borrower token fetching another borrower’s record, a lender token fetching an SHG it is not linked to) and correctly rejected with 401/403 in every case tested.

5.4 Bugs Found and Fixed

Several real defects were found by running the system end-to-end (training on real data, hitting the live API, and exercising the live frontend) rather than by reading the code:

1. A hard `missed_fraction > 0.4` cliff made the synthetic default label a near coin-flip at the boundary – replaced with a smooth, probabilistic default rule.
2. SHAP explanations were allocated as a share of `score - floor`, which silently went nearly blank for borrowers scoring near the floor – exactly the people who most need an explanation. Fixed by anchoring on the model’s own baseline prediction instead.
3. An SHG-lender link’s `requested_date` could land in the future for a recently formed SHG – capped at today.
4. k-means cluster labels were inverted (worst cluster labelled ”High Trust”).
5. Missing indexes on every foreign key, and missing `__init__.py` files in three subpackages.
6. An unbounded eligible-borrowers table rendered 351 rows with no cap – capped at 25 with a ”showing top N of M” note.
7. A frontend/backend pagination mismatch made roughly half the population unreachable by search – replaced with real server-side search.
8. The ”Approve/Reject Partnership” buttons on the Lender and Admin dashboards referenced a field name (`link_id`) that does not exist in the API response (the real field is `id`) – every click silently called the decide endpoint with `undefined`, meaning a lender could never actually approve or reject an SHG link through the UI. Fixed and verified live.
9. `GET /api/shgs` computed each SHG’s aggregate repayment/attendance rate as a Python property walking `members -> loans -> repayment_events` per SHG – an N+1 pattern materializing on the order of 25,000+ ORM objects per request against the real dataset, measured at 1,101–1,322 ms median. Replaced with three `GROUP BY SQL` aggregates: 14.5–26.7 ms median, a $\sim 75\times$ improvement, with identical response content verified before and after.

6 Conclusion and Future Scope

...where this stands and what’s next

At the prototype stage, CreditSetu has a working, explainable, dual-path scoring engine; a formal SHG-lender linkage; a two-directional loan marketplace with targeted requests; four separately authorized role-based views with server-side access control; and a live borrower repayment flow whose impact on credit score has been verified against a real account, not just claimed. Real defects, including one that silently broke a core lender action and one with a measured $75\times$ performance impact, were found and fixed by exercising the live system rather than only reading the code.

Remaining work toward the Final Report: periodic re-scoring so genuine score-history trends accumulate over time (currently limited by how few times the batch-scoring pipeline has actually

been run); admin account-management/moderation actions beyond the current read-only oversight view; and replacing the deliberately minimal demo bearer-token/password scheme with production-grade hashing and short-lived signed tokens. The independent-vs-SHG base-score gap remains a stated policy parameter rather than something the data derived, and is flagged as a candidate for a fairness sensitivity analysis in the Final Report.

References

- [1] Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

A Use Case Diagram

Four actors (Borrower, Lender, SHG, Admin) against every real use case grounded in the actual API routers and dashboards. Full written use-case scenarios (preconditions, main/alternate flows, postconditions) for the six most important use cases are in `docs/uml/use-case-diagram.md` in the repository, alongside the PlantUML source (regenerate/edit at <https://www.plantuml.com/plantuml/>).

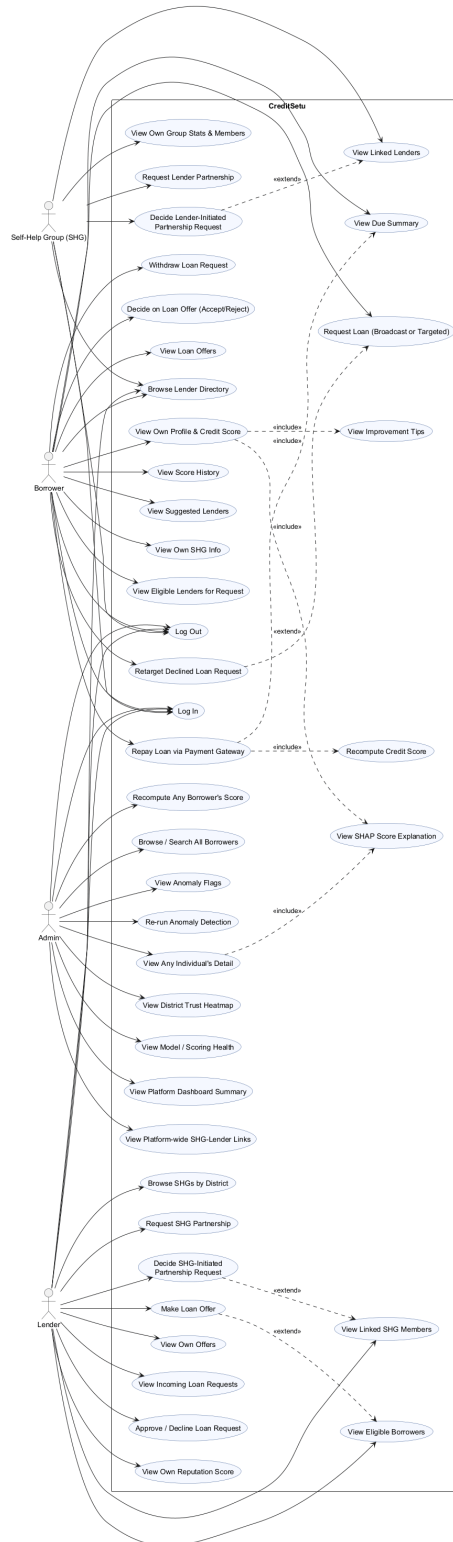


Figure 1: CreditSetu use case diagram

B Class Diagram

The real SQLAlchemy schema (`app/models.py`), including the role-based-access session tables and the payments-feature additions. See `docs/uml/class-diagram.md` for the PlantUML source and the accompanying design-decision notes (the nullable `shg_id` dual-path pivot, append-only `CreditScore` history, and the `SHGLenderLink` many-to-many approve/reject table).

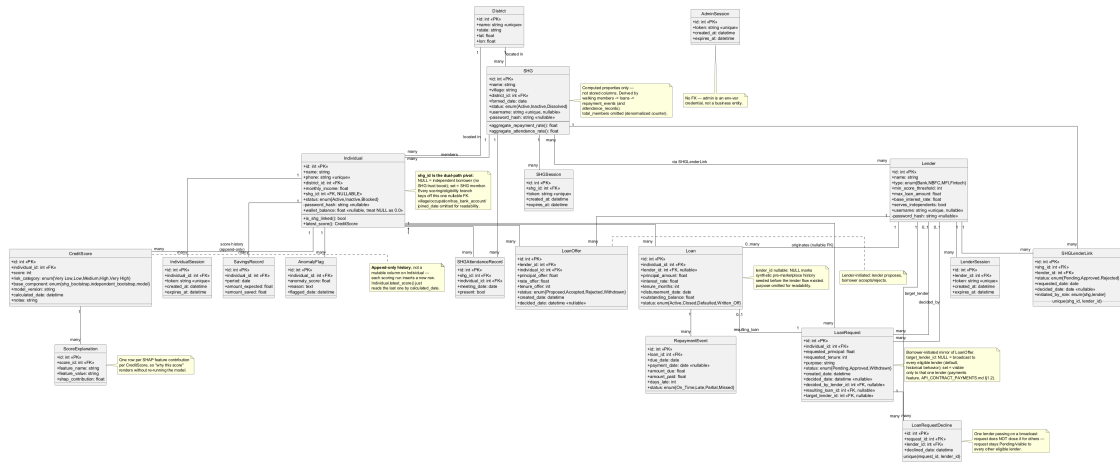


Figure 2: CreditSetu class diagram

C Activity Diagram

The real request → approval → repayment → rescoring control flow, with Borrower / System (Backend) / Lender swimlanes, including the aggregate insufficient-funds gate and the race-condition guard on the wallet balance. See `docs/uml/activity-diagram.md` for the PlantUML source and a step-by-step prose walkthrough mapping each step to the real backend function it corresponds to.

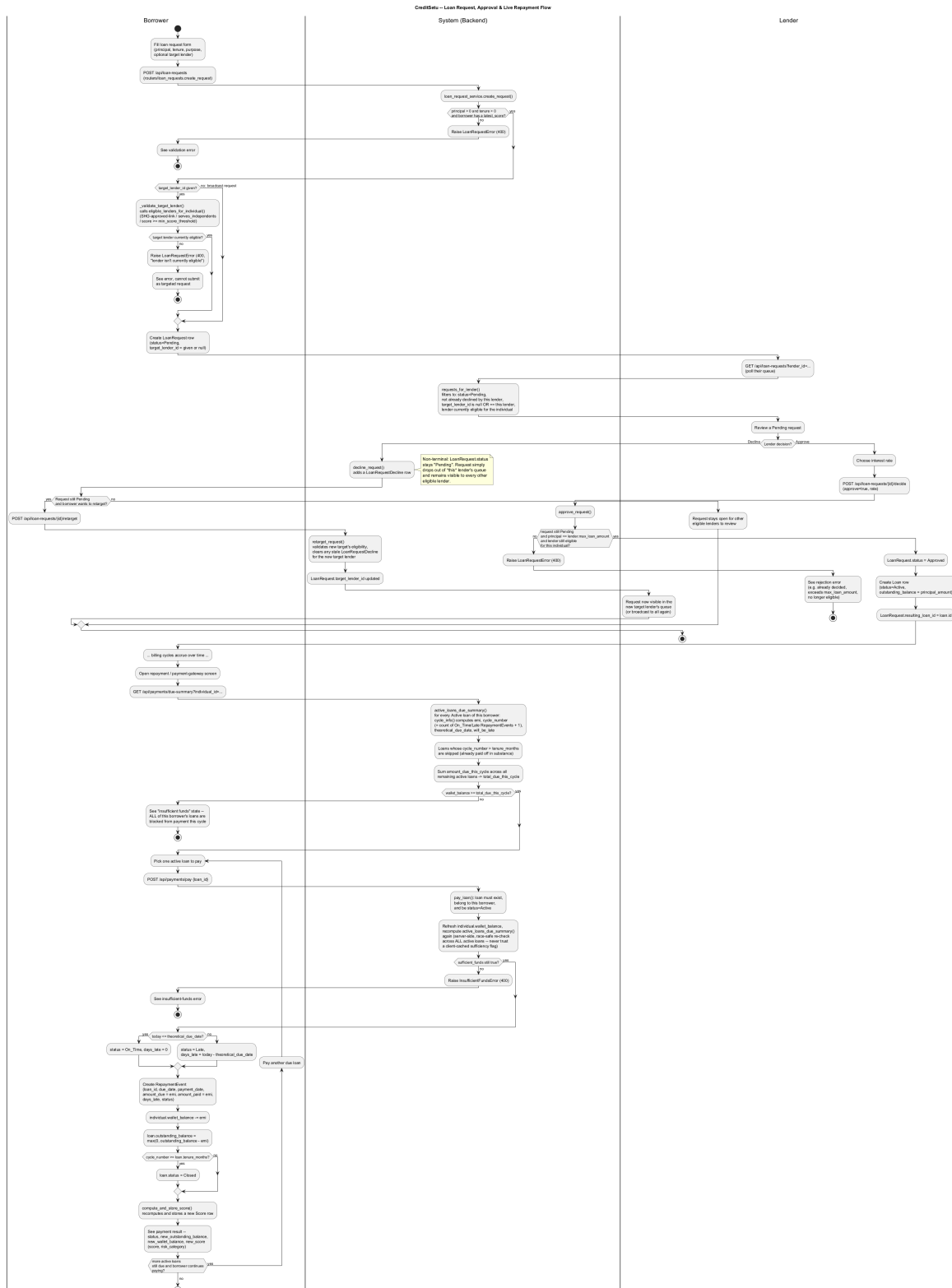


Figure 3: CreditSetu activity diagram: loan request, approval, and live repayment